S. B. JAIN INSTITUTE OF TECHNOLOGY,

MANAGEMENT & RESEARCH, NAGPUR.

Practical No. 4

Aim: To build and evaluate a Decision Tree model for classification tasks and study the effect of tree depth and splitting criteria.

Name of Student : _____

Roll No. : _____

Semester/Year : _____

Academic Session : _____

Date of Performance : _____

Date of Submission : _____

AIM: To build and evaluate a Decision Tree model for classification tasks and study the effect of tree depth and splitting criteria.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- It allows an individual or organization to weigh possible actions against one another based on their costs, probabilities, and benefits.
- A decision tree is a map of the possible outcomes of a series of related choices. ● A decision model provides a way to visualize the sequences of events that can occur following alternative decisions (or actions) in a logical framework.
- An objective decision is one which is not influenced by one's personal feelings, perspectives, interests and biases.
- A decision tree is a map of the possible outcomes of a series of related choices. It allows an individual or organization to weigh possible actions against one another based on their costs, probabilities, and benefits.

THEORY:

DECISION TREE:-

A decision tree is a type of supervised machine learning used to categorize or make predictions based on how a previous set of questions were answered. The model is a form of supervised learning, meaning that the model is trained and tested on a set of data that contains the desired categorization.

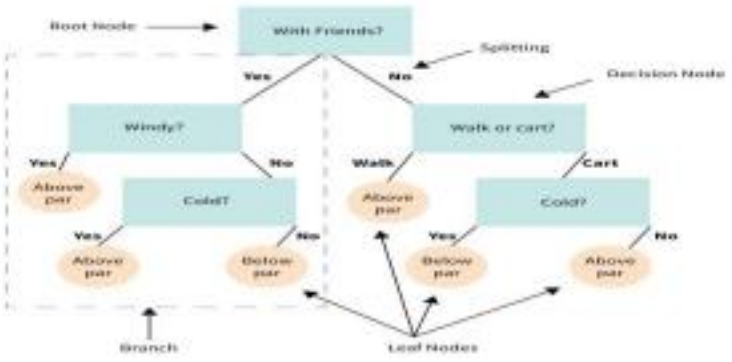
The decision tree may not always provide a clear-cut answer or decision. Instead, it may present options so the data scientist can make an informed decision on their own. Decision trees imitate human thinking, so it‟s generally easy for data scientists to understand and interpret the results.

How Does the Decision Tree Work?

Some key terms of a decision tree are as follows:.

- **Root node:** The base of the decision tree.
- **Splitting:** The process of dividing a node into multiple sub-nodes.
- **Decision node:** When a sub-node is further split into additional sub-nodes.
- **Leaf node:** When a sub-node does not further split into additional sub-nodes; represents possible outcomes.
- **Pruning:** The process of removing sub-nodes of a decision tree.
- **Branch:** A subsection of the decision tree consisting of multiple nodes.

A decision tree resembles, well, a tree. The base of the tree is the root node. From the root node flows a series of decision nodes that depict decisions to be made. From the decision nodes are leaf nodes that represent the consequences of those decisions. Each decision node represents a question or split point, and the leaf nodes that stem from a decision node represent the possible answers. Leaf nodes sprout from decision nodes similar to how a leaf sprouts on a tree branch. This is why we call each subsection of a decision tree a "branch." Let"s take a look at an example for this. You"re a golfer, and a consistent one at that. On any given day you want to predict where your score will be in two buckets: below par or over par.



There are two main types of decision treesExternal link:

Categorical Variable Decision Tree

In a categorical variable decision tree, the answer neatly fits into one category or another. Was the coin toss heads or tails? Is the animal a reptile or mammal? In this type of decision tree, data

is placed into a single category based on the decisions at the nodes throughout the tree.

Continuous Variable Decision Tree

A continuous variable decision tree is one where there is not a simple yes or no answer. It"s also known as a regression tree because the decision or outcome variable depends on other decisions farther up the tree or the type of choice involved in the decision. The benefit of a continuous variable decision tree is that the outcome can be predicted based on multiple variables rather than on a single variable as in a categorical variable decision tree. Continuous variable decision trees are used to create predictions. The system can be used for both linear and non-linear relationships if the correct algorithm is selected.

The algorithm selection is also based on the type of target variables. Let us look at some algorithms used in Decision Trees:

ID3 → (extension of D3)

C4.5 → (successor of ID3)

CART → (Classification And Regression Tree)

CHAID → (Chi-square automatic interaction detection Performs multi-level splits when computing classification trees)

MARS → (multivariate adaptive regression splines)

The ID3 algorithm builds decision trees using a top-down greedy search approach through the space of possible branches with no backtracking. A greedy algorithm, as the name suggests, always makes the choice that seems to be the best at that moment.

Steps in ID3 algorithm:

1. It begins with the original set S as the root node.
2. On each iteration of the algorithm, it iterates through the very unused attribute of the set S and calculates **Entropy(H)** and **Information gain(IG)** of this attribute.
3. It then selects the attribute which has the smallest Entropy or Largest Information gain.
4. The set S is then split by the selected attribute to produce a subset of the data. 5. The algorithm continues to recur on each subset, considering only attributes never selected before.

For solving this attribute selection problem, researchers worked and devised some solutions. They suggested using some *criteria* like :

Entropy,

Information gain,

Gini index,

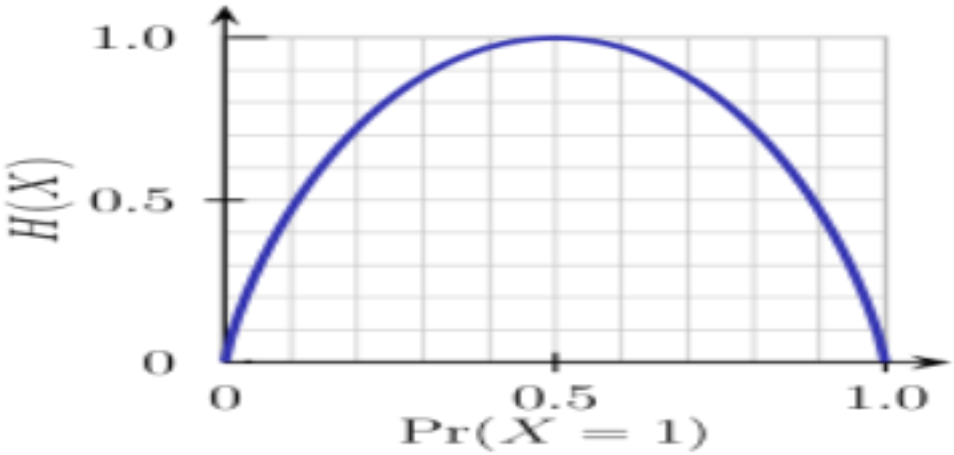
Gain Ratio,

Reduction in Variance

Chi-Square

Entropy

Entropy is a measure of the randomness in the information being processed. The higher the entropy, the harder it is to draw any conclusions from that information. Flipping a coin is an example of an action that provides information that is random.



ID3 follows the rule — A branch with an entropy of zero is a leaf node and A brach with entropy more than zero needs further splitting.

Mathematically Entropy for 1 attribute is represented as:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5

Entropy(PlayGolf) = Entropy (5,9)
= Entropy (0.36, 0.64)
= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)
= 0.94

Gini Index

You can understand the Gini index as a cost function used to evaluate splits in the dataset. It is calculated by subtracting the sum of the squared probabilities of each class from one. It favors larger partitions and easy to implement whereas information gain favors smaller partitions with distinct values.

$$Gini = 1 - \sum_{i=1}^{C} (p_i)^2$$

we have a dataset that has users and their movie genre preferences based on variables like gender, group of age, rating, blah, blah. With the help of information gain, you split at „Gender" (assuming it has the highest information gain) and now the variables „Group of Age" and „Rating" could be equally important and with the help of gain ratio, it will penalize a variable with more distinct values which will help us decide the split at the next level.

$$Gain\ Ratio = \frac{Information\ Gain}{SplitInfo} = \frac{Entropy\ (before) - \sum_{j=1}^{K} Entropy(j, after)}{\sum_{j=1}^{K} w_j \log_2 w_j}$$

PROGRAM CODE:

OUTPUT (SCREENSHOT):

```
# Prathamesh (CS23115)
# AIM to build and evaluate a decision tree model for classfication  tasks and study t
```

```
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

```
path="/content/drive/MyDrive/ML_DATASET/daily_weather.csv"
df=pd.read_csv(path)
df.head(5)
```

	number	air_pressure_9am	air_temp_9am	avg_wind_direction_9am	avg_wind_speed_9am
0	0	918.060000	74.822000	271.100000	2.080354
1	1	917.347688	71.403843	101.935179	2.443009
2	2	923.040000	60.638000	51.000000	17.067852
3	3	920.502751	70.138895	198.832133	4.337363
4	4	921.160000	44.294000	277.800000	1.856660

```
df.info()
```

```
df.shape
```

```
(1095, 11)
```

```
df.describe()
```

	number	air_pressure_9am	air_temp_9am	avg_wind_direction_9am	avg_wind_sp
count	1095.000000	1092.000000	1090.000000	1091.000000	1092
mean	547.000000	918.882551	64.933001	142.235511	5
std	316.243577	3.184161	11.175514	69.137859	4
min	0.000000	907.990000	36.752000	15.500000	0
25%	273.500000	916.550000	57.281000	65.972506	2
50%	547.000000	918.921045	65.715479	166.000000	3
75%	820.500000	921.160073	73.450974	191.000000	7
max	1094.000000	929.320000	98.906000	343.400000	23

```
df.columns
```

```
Index(['number', 'air_pressure_9am', 'air_temp_9am', 'avg_wind_direction_9am',
       'avg_wind_speed_9am', 'max_wind_direction_9am', 'max_wind_speed_9am',
       'rain_accumulation_9am', 'rain_duration_9am', 'relative_humidity_9am',
       'relative_humidity_3pm'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.True_
```

```
df.isnull().sum()
```

	0
number	0
air_pressure_9am	3
air_temp_9am	5
avg_wind_direction_9am	4
avg_wind_speed_9am	3
max_wind_direction_9am	3
max_wind_speed_9am	4
rain_accumulation_9am	6
rain_duration_9am	3
relative_humidity_9am	0
relative_humidity_3pm	0

dtype: int64

```
#print missing value
df[df.isnull().any(axis=1)]
```

	number	air_pressure_9am	air_temp_9am	avg_wind_direction_9am	avg_wind_speed_
16	16	917.890000	NaN	169.200000	2.192
111	111	915.290000	58.820000	182.600000	15.613
177	177	915.900000	NaN	183.300000	4.719
262	262	923.596607	58.380598	47.737753	10.636
277	277	920.480000	62.600000	194.400000	2.751
334	334	916.230000	75.740000	149.100000	2.751
358	358	917.440000	58.514000	55.100000	10.021
361	361	920.444946	65.801845	49.823346	21.520
381	381	918.480000	66.542000	90.900000	3.467
409	409	NaN	67.853833	65.880616	4.328
517	517	920.570000	53.600000	100.100000	4.697
519	519	916.250000	55.670000	176.400000	6.666
546	546	NaN	42.746000	251.100000	12.929
620	620	921.200000	56.786000	192.300000	9.551
625	625	912.400000	50.774000	171.600000	N
656	656	920.830000	66.344000	NaN	15.457
670	670	910.920000	48.362000	156.500000	N
672	672	922.448945	72.863773	NaN	3.682
705	705	911.900000	59.072000	199.800000	1.275

```
df.drop('number', axis=1, inplace=True, errors='ignore')
df.columns
```

```
Index(['air_pressure_9am', 'air_temp_9am', 'avg_wind_direction_9am',
       'avg_wind_speed_9am', 'max_wind_direction_9am', 'max_wind_speed_9am',
       'rain_accumulation_9am', 'rain_duration_9am', 'relative_humidity_9am',
       'relative_humidity_3pm'],
      dtype='object')
```

```
df.shape
```

```
(1095, 10)
```

```
before_rows=df.shape[0]
df = df.dropna()
after_rows = df.shape[0]
print("Before rows:",before_rows)
print("After rows:",after_rows)
print("The number of dropped rows are:",before_rows-after_rows)
```

```
Before rows: 1095
After rows: 1064
The number of dropped rows are: 31
```

```
df.shape
```

```
(1064, 10)
```

```
clean_data = df.copy()
clean_data ['high_humidity_label'] = (clean_data['relative_humidity_3pm']>24.99)*1
print(clean_data['high_humidity_label'])
```

```
0       1
1       0
2       0
3       0
4       1
       ..
1090    1
1091    1
1092    1
1093    1
1094    0
Name: high_humidity_label, Length: 1064, dtype: int64
```

```
clean_data.columns
```

```
Index(['air_pressure_9am', 'air_temp_9am', 'avg_wind_direction_9am',
       'avg_wind_speed_9am', 'max_wind_direction_9am', 'max_wind_speed_9am',
       'rain_accumulation_9am', 'rain_duration_9am', 'relative_humidity_9am',
       'relative_humidity_3pm', 'high_humidity_label'],
      dtype='object')
```

```
clean_data.drop('relative_humidity_9am', axis=1, inplace=True, errors='ignore')
clean_data.drop('relative_humidity_3pm', axis=1, inplace=True, errors='ignore')
clean_data.columns
```

```
Index(['air_pressure_9am', 'air_temp_9am', 'avg_wind_direction_9am',
       'avg_wind_speed_9am', 'max_wind_direction_9am', 'max_wind_speed_9am',
       'rain_accumulation_9am', 'rain_duration_9am', 'high_humidity_label'],
      dtype='object')
```

```python
X=clean_data.iloc[:,:-1]
print(X)
```

```
      air_pressure_9am  air_temp_9am  avg_wind_direction_9am  \
0           918.060000     74.822000              271.100000
1           917.347688     71.403843              101.935179
2           923.040000     60.638000               51.000000
3           920.502751     70.138895              198.832133
4           921.160000     44.294000              277.800000
...                ...           ...                     ...
1090        918.900000     63.104000              192.900000
1091        918.710000     49.568000              241.600000
1092        916.600000     71.096000              189.300000
1093        912.600000     58.406000              172.700000
1094        921.530000     77.702000               97.100000

      avg_wind_speed_9am  max_wind_direction_9am  max_wind_speed_9am  \
0               2.080354              295.400000            2.863283
1               2.443009              140.471548            3.533324
2              17.067852               63.700000           22.100967
3               4.337363              211.203341            5.190045
4               1.856660              136.500000            2.863283
...                  ...                     ...                 ...
1090            3.869906              207.300000            5.212070
1091            1.811921              227.400000            2.371156
1092            3.064608              200.800000            3.892276
1093            3.825167              189.100000            4.764682
1094            3.265932              125.900000            4.451511

      rain_accumulation_9am  rain_duration_9am
0                       0.0                0.0
1                       0.0                0.0
2                       0.0               20.0
3                       0.0                0.0
4                       8.9            14730.0
...                     ...                ...
1090                    0.0                0.0
1091                    0.0                0.0
1092                    0.0                0.0
1093                    0.0                0.0
1094                    0.0                0.0

[1064 rows x 8 columns]
```

```python
Y=clean_data.iloc[:,-1]
print(Y)
```

```
0       1
1       0
2       0
3       0
4       1
       ..
1090    1
1091    1
1092    1
1093    1
1094    0
Name: high_humidity_label, Length: 1064, dtype: int64
```

```python
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.3, random_state=
```

```
X_train
```

	air_pressure_9am	air_temp_9am	avg_wind_direction_9am	avg_wind_speed_9am	max_w
5	915.30000	78.404000	182.800000	9.932014	
315	917.09000	63.266000	175.500000	6.733189	
918	915.07000	58.046000	186.200000	7.695074	
105	917.40000	73.868000	181.700000	4.138339	
98	916.52000	51.638000	190.800000	7.404271	
...	...	...	...	...	
336	912.89000	45.320000	213.400000	21.250930	
476	917.18000	62.834000	186.300000	3.221194	
123	920.18000	62.744000	32.600000	3.019869	
1075	922.85811	64.989361	63.483047	10.261187	
886	918.70000	69.422000	161.100000	0.782929	

744 rows × 8 columns

```
X_test
```

	air_pressure_9am	air_temp_9am	avg_wind_direction_9am	avg_wind_speed_9am	max_w
32	918.700000	76.226000	184.600000	4.093600	
1084	917.130000	80.240000	183.300000	1.632966	
423	921.090000	70.124000	182.800000	2.908022	
767	915.220000	73.292000	148.300000	1.789552	
818	921.400000	63.050000	223.200000	1.319795	
...	...	...	...	...	
503	919.738161	67.432744	147.470882	1.157775	
585	916.250000	79.754000	48.600000	1.073731	
389	916.500000	74.588000	101.200000	1.834291	
715	923.600000	65.030000	62.300000	7.091100	
352	925.280000	66.470000	49.400000	11.229439	

320 rows × 8 columns

```
Y_train
```

	high_humidity_label
5	1
315	1
918	1
105	1
98	1
...	...
336	1
476	1
123	1
1075	0
886	0

744 rows × 1 columns

dtype: int64

```
Y_test
```

	high_humidity_label
32	1
1084	1
423	1
767	1
818	1
...	...
503	0
585	1
389	1
715	1
352	0

320 rows × 1 columns

dtype: int64

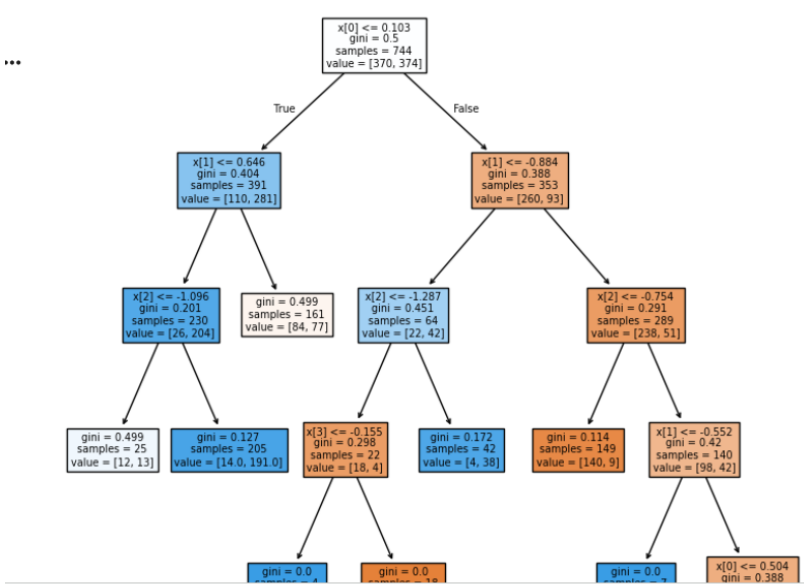
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler = scaler.transform(X_test)
```

```
humidity_classifier = DecisionTreeClassifier(max_leaf_nodes=10, random_state=0)
humidity_classifier.fit(X_train, Y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_leaf_nodes=10, random_state=0)
```

```
import matplotlib.pyplot as plt
from sklearn import tree
plt.figure(figsize=(10,10))
tree.plot_tree(humidity_classifier, filled=True)
```

```
[Text(0.39285714285714285, 0.9166666666666666, 'x[0] <= 0.103\ngini = 0.5\nsamples =
744\nvalue = [370, 374]'),
 Text(0.21428571428571427, 0.75, 'x[1] <= 0.646\ngini = 0.404\nsamples = 391\nvalue =
[110, 281]'),
 Text(0.30357142857142855, 0.8333333333333333, 'True  '),
 Text(0.14285714285714285, 0.5833333333333334, 'x[2] <= -1.096\ngini = 0.201\nsamples
= 230\nvalue = [26, 204]'),
 Text(0.07142857142857142, 0.4166666666666667, 'gini = 0.499\nsamples = 25\nvalue =
[12, 13]'),
 Text(0.21428571428571427, 0.4166666666666667, 'gini = 0.127\nsamples = 205\nvalue =
[14.0, 191.0]'),
 Text(0.2857142857142857, 0.5833333333333334, 'gini = 0.499\nsamples = 161\nvalue =
[84, 77]'),
 Text(0.5714285714285714, 0.75, 'x[1] <= -0.884\ngini = 0.388\nsamples = 353\nvalue =
[260, 93]'),
 Text(0.4821428571428571, 0.8333333333333333, '  False'),
 Text(0.42857142857142855, 0.5833333333333334, 'x[2] <= -1.287\ngini = 0.451\nsamples
= 64\nvalue = [22, 42]'),
 Text(0.35714285714285715, 0.4166666666666667, 'x[3] <= -0.155\ngini = 0.298\nsamples
= 22\nvalue = [18, 4]'),
 Text(0.2857142857142857, 0.25, 'gini = 0.0\nsamples = 4\nvalue = [0, 4]'),
 Text(0.42857142857142855, 0.25, 'gini = 0.0\nsamples = 18\nvalue = [18, 0]'),
 Text(0.5, 0.4166666666666667, 'gini = 0.172\nsamples = 42\nvalue = [4, 38]'),
 Text(0.7142857142857143, 0.5833333333333334, 'x[2] <= -0.754\ngini = 0.291\nsamples
= 289\nvalue = [238, 51]'),
 Text(0.6428571428571429, 0.4166666666666667, 'gini = 0.114\nsamples = 149\nvalue =
[140, 9]'),
 Text(0.7857142857142857, 0.4166666666666667, 'x[1] <= -0.552\ngini = 0.42\nsamples =
140\nvalue = [98, 42]'),
 Text(0.7142857142857143, 0.25, 'gini = 0.0\nsamples = 7\nvalue = [0, 7]'),
 Text(0.8571428571428571, 0.25, 'x[0] <= 0.504\ngini = 0.388\nsamples = 133\nvalue =
[98, 35]'),
 Text(0.7857142857142857, 0.08333333333333333, 'gini = 0.497\nsamples = 54\nvalue =
[29.0, 25.0]'),
 Text(0.9285714285714286, 0.08333333333333333, 'gini = 0.221\nsamples = 79\nvalue =
[69, 10]')]
```




```
Y_pred = humidity_classifier.predict(X_test)
print(Y_pred)
```

```
[0 0 0 0 0 1 0 1 0 1 0 1 0 1 0 1 1 0 0 1 1 0 1 1 1 0 0 0 0 0 0 0 1 0 0 0 1
 0 1 0 1 1 1 0 1 1 0 0 1 1 1 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 1 1
 1 1 0 0 1 1 1 0 0 1 0 0 0 1 1 0 0 1 0 0 1 0 0 0 0 0 0 0 1 1 1 1 0 0 1 0 0
 1 1 1 1 1 0 0 0 0 0 1 1 1 0 1 0 1 0 1 0 1 0 0 1 1 0 0 1 0 0 1 1 1 1 1 0
 0 0 0 0 1 0 1 0 0 0 1 1 1 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 0 1 1 0 0 1 0 0 0
 0 0 1 0 0 1 0 1 0 1 0 1 1 1 1 0 1 0 1 0 0 0 1 1 1 0 0 1 1 0 1 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 0 0 0 1 1 0 1 1 0 0 0 1 0 0 0 1 1 0 0 1 0 0 1 0 0 0 0 0 1
 0 1 1 0 1 0 1 1 1 0 0 1 0 0 0 0 0 1 0 1 0 1 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0
 1 1 1 1 0 1 0 1 0 1 0 0 0 1 1 0 1 0 0 0 0 0 0 0]
```

```
#Accuraccy
from sklearn.metrics import confusion_matrix, accuracy_score,classification_report
```

```
print(confusion_matrix(Y_test, Y_pred))
```

```
[[138  27]
 [ 52 103]]
```

```
print(classification_report(Y_test, Y_pred))
```

```
              precision    recall  f1-score   support

           0       0.73      0.84      0.78       165
           1       0.79      0.66      0.72       155

    accuracy                           0.75       320
   macro avg       0.76      0.75      0.75       320
weighted avg       0.76      0.75      0.75       320
```

CONCLUSION:

_____

_____

_____

_____

DISCUSSION AND VIVA VOCE:

1. When would you prefer a Decision Tree over Logistic Regression in a real-world project?

Ans :-

2. How do you detect and control overfitting in a Decision Tree model deployed in production?

Ans :-

3. Why Decision Trees are considered highly interpretable, and how is this useful in regulated industries like banking or healthcare?

Ans :-

4. What happens if your Decision Tree is allowed to grow without depth restriction?

Ans :-

5. How do you decide the optimal value of max_depth in an industry ML pipeline?

Ans :-

6. Why don"t Decision Trees require feature scaling, and when might scaling still be necessary?

Ans :-

7. How do Decision Trees handle categorical variables in real datasets?

Ans :-

8. What are the limitations of Decision Trees in high-dimensional datasets?

Ans :-

9. How do you evaluate whether a Decision Tree model is stable and reliable before deployment?

Ans :-

10. Why are ensemble methods like Random Forest often preferred over a single Decision Tree in production systems?

Ans :-

REFERENCE

- https://www.mastersindatascience.org/learning/machine-learning-algorithms/decision tree/#:~:text=A%20decision%20tree%20is%20a,that%20contains%20the%20desired